

Garage Automation System

This project envisions a **smart underground garage** designed for a futuristic and secure residential environment. The system automates the process of retrieving and storing vehicles, making it highly efficient and user-friendly. Imagine it as an intelligent car storage and management system, similar to how car companies handle their vehicles in advanced storage facilities. The goal is to ensure the **right car is delivered to the right person at the right time** with minimal human intervention.

Why Build This Automated Garage?

1. **Enhanced Security:**
The system provides higher levels of security for cars, reducing the risk of theft or unauthorized access. Both residents and visitors can benefit from this feature.
 2. **Futuristic Design:**
It aligns with modern architectural trends, contributing to a **smart residence** concept that incorporates advanced technologies for daily convenience.
 3. **Additional Services:**
Beyond storing and retrieving cars, the system could be extended to offer **car cleaning services, status analysis (e.g., fuel level, maintenance needs)**, or even evaluate the **market value** of cars.
 4. **Efficient Use of Space:**
An underground, automated system can save significant space compared to traditional garages, making it ideal for urban environments.
-

How the System Works

The flow can be described as follows:

1. **Pattern :**
 - The system will manage data hierarchically, connecting cars to individual owners or residents in a specific house.
 - A request is sent (e.g., by a resident or visitor) to the system, which identifies and retrieves the corresponding car from the garage.
2. **Functionalities:**
 - **Retrieve Cars:** The system retrieves a specific car using automated mechanisms and presents it to the user.
 - **Choose the Car:** If there are multiple cars associated with a user, the system lets them select the desired one.
 - **Identify the User:** The system recognizes the user (e.g., via ID).

- **Calculate Timing:** It estimates the time required to bring the car based on its position in the garage.
-

Object-Oriented Programming (OOP) Concepts Used

1. Inheritance:

- The structure of classes will share common properties and methods using inheritance.

2. Polymorphism:

- Methods like adding for users or cars will be flexible, retrieving, alert .

3. Abstract Base Class (ABC):

- The **assets Class** acts as an abstract class.

4. Destructors:

- Destructors (retrieve() method) can be used for cleaning up resources after car retrieval tasks are completed, ensuring efficient memory usage.

5. Composition:

- For example between assets class and other classes.